



UNIVERSITY PARTNER



Artificial Intelligence and Machine Learning — 6CS012

Final Portfolio Assessment 2026 | Part III: Language
Task

Submitted By : Dipesh Sherchan

Student Id : 2438455

Group : L6CG7

Submission Date : 2026/05/10

Abstract

The report describes a complete natural language processing (NLP) system which processes short news headlines to produce five different category results which include Sports Tech Money Culture & Arts Education and Education. The dataset consists of 11,025 labelled headlines which were collected from a news aggregation corpus. The researchers created and executed three deep learning models which included a Simple Recurrent Neural Network (SimpleRNN) and a Bidirectional Long Short-Term Memory (BiLSTM) network that used a trainable embedding layer and a BiLSTM network that integrated pretrained Word2Vec embeddings which were developed through Gensim using the skip-gram algorithm. The text preprocessing process used a standard pipeline which included converting text to lowercase and solving contractions and removing URLs and mentions and hashtags and filtering out stop words and performing lemmatization. The Keras Tokenizer executed tokenization process which used a vocabulary base of 10,000 tokens to convert sequences into fixed-length sequences of 11 tokens which represented the 95th percentile of training sequence lengths. The SimpleRNN achieved test accuracy of 71.79% while the BiLSTM reached 82.59% and the BiLSTM with Word2Vec embeddings reached top performance of 83.90%. The results prove that stacked bidirectional LSTM layers with pretrained contextual embeddings produce better results than standard RNN methods when used on short-text classification tasks. The project also includes an interactive Gradio web interface which enables users to predict headlines in real time.

1. Introduction

Natural Language Processing uses text classification as a basic task which helps machines to automatically determine pre-set categories for unstructured text. The classification of news headlines creates important practical use cases which include content recommendation systems news aggregation platforms automated content moderation and search engine indexing.

Text classification processes employ traditional machine learning methods which include Naive Bayes and Support Vector Machines. The methods depend on manually created features which do not manage to preserve the language's sequential dependencies. Recurrent Neural Networks (RNNs) and their improved variant, Long Short-Term Memory (LSTM) networks, offer a more principled approach by processing text as sequential data and learning contextual dependencies across tokens.

The SimpleRNNs limitation on longer sequences is solved by LSTM units through their three-gate system which includes input gates and forget gates and output gates. The use of pretrained word embeddings such as Word2Vec enables models to begin with semantically rich word representations which helps them achieve faster outcomes while better performance on smaller datasets.

This research project examines three distinct RNN-based architectures to assess their effectiveness in multi-class headline classification. The researchers conducted their experiments by training their models to handle 11,025 news headlines which they divided into five different categories. The report presents the entire process which includes text preprocessing and tokenization and model architecture design and model training and evaluation and error analysis according to the 6CS012 Final Portfolio Assessment 2026 guidelines.

2. Dataset

2.1. Source

The used dataset is news_category.csv, a structured text dataset containing 'News headlines' paired with their related category labels.

2.2. Data and Structure

Property	Value
Total Records	11,025
Number of Features	2 (headline, category)
Number of Classes	5
Missing Values	None
Data Types	Both columns: object (string)

2.3. Class Distribution

Category	Count	Proportion
SPORTS	5,077	46.1%
TECH	2,104	19.1%
MONEY	1,756	15.9%
CULTURE & ARTS	1,074	9.7%
EDUCATION	1,014	9.2%

```
Dimensions : (11025, 2)
Columns : ['headline', 'category']
```

	headline	category
0	How A New Documentary Captures The Complexity ...	CULTURE & ARTS
1	Twitch Bans Gambling Sites After Streamer Scam...	TECH
2	'Reboot' Is A Clever And Not Too Navel-Gazey L...	CULTURE & ARTS
3	Maury Wills, Base-Stealing Shortstop For Dodge...	SPORTS
4	Las Vegas Aces Win First WNBA Title, Chelsea G...	SPORTS
5	Meet Alex Aster, The TikTokker Changing The Pub...	CULTURE & ARTS
6	TikTok Search Results Riddled With Misinformat...	TECH
7	Boston Marathon To Make Race More Inclusive Fo...	SPORTS
8	Anthony Varvaro, MLB Pitcher Turned Transit Co...	SPORTS
9	How Ani Liu Is Brilliantly Disguising Her Art ...	CULTURE & ARTS

Figure 1: output of news_df.head showing the first 10 rows

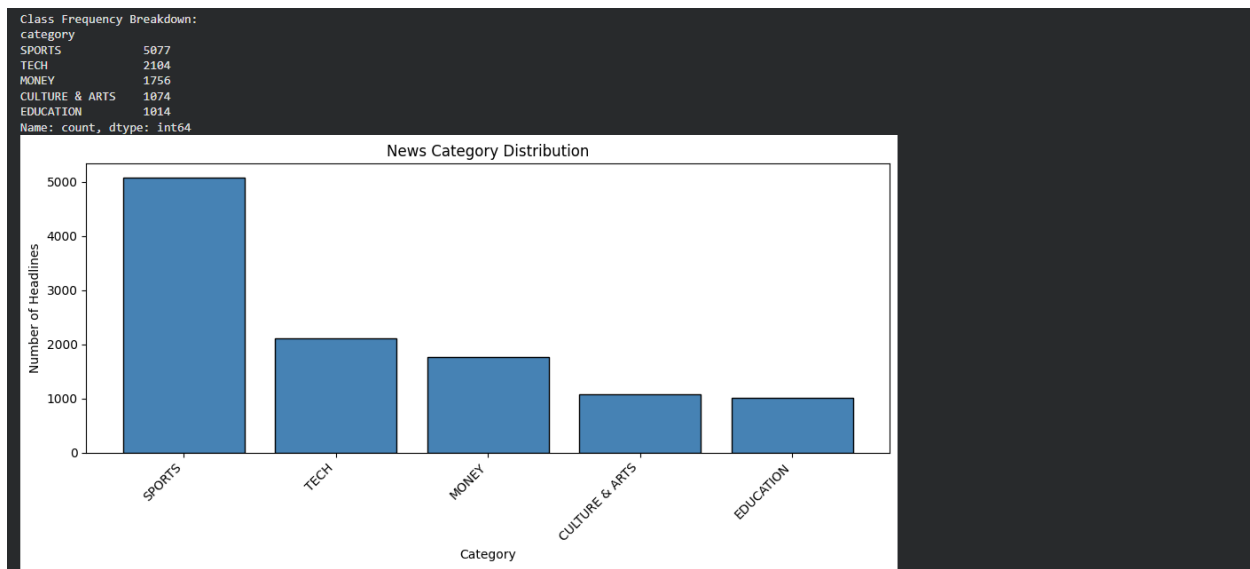


Figure 2: class distribution bar chart

2.4. Preprocessing Steps

Each raw headline was processed through the following sequential steps before tokenization:

1. **Lowercasing:** Every characters are converted to lowercase for uniformity.
2. **Contraction resolution:** For e.g., "don't" → "do not" by using a predefined 30 entry map.

3. **(URL, mention, hashtag, number) removal** — regex patterns strip http://..., @user, #tag, digits.
4. **Special-Character removal:** non-alphanumeric or non-space characters are removed.
5. **Stop words removal:** English stopwords(it, the, etc.) are filtered using NLTK's stopwords corpus.
6. **Lemmatization:** words are reduced to root form using NLTK's WordNetLemmatizer.

Sample Preprocessing Results:

Raw Headline	Processed
How A New Documentary Captures The Complexity Of Being A Child Of Immigrants	new documentary capture complexity child immigrant
Twitch Bans Gambling Sites After Streamer Scams Folks Out Of \$200,000	twitch ban gambling site streamer scam folk
'Reboot' Is A Clever And Not Too Navel-Gazey Look Inside TV Reboots	reboot clever navelgazey look inside tv reboots
Maury Wills, Base-Stealing Shortstop For Dodgers, Dies At 89	maury will basestealing shortstop dodger dy
Las Vegas Aces Win First WNBA Title, Chelsea Gray Named MVP	la vega ace win first wnba title chelsea gray named mvp

3. Methodology

3.1. Tokenization and Sequence Padding

The data became ready for model input after the process of text cleaning was completed.:

- **Train/Test Split:** 80% training (8,820 samples) / 20% testing (2,205 samples), stratified by category label.
- **Tokenization:** The Keras Tokenizer was used for tokenization with a vocabulary size of 10000 words and an (out-of-vocabulary) token.
- **Unique vocabulary tokens discovered:** 12,036 (top 10,000 retained).
- **Sequence padding:** pad_sequences with padding='post', truncating='post', MAX_LEN = 11.

MAX_LEN = 11 was chosen as the **95th percentile** of training sequence lengths, balancing information retention against unnecessary padding overhead.

Token Sequence Length Statistics (Training Set):

Statistic	Value
Minimum	1
Mean	6.9
Median	7.0
95th Percentile (MAX_LEN)	11
Maximum	18

Padded Array Shapes:

- Train: (8820, 11)
- Test: (2205, 11)

```
Token Sequence Length Statistics:
Minimum : 1
Mean : 6.9
Median : 7.0
95th pct : 11 <- chosen as MAX_LEN
Maximum : 18
```

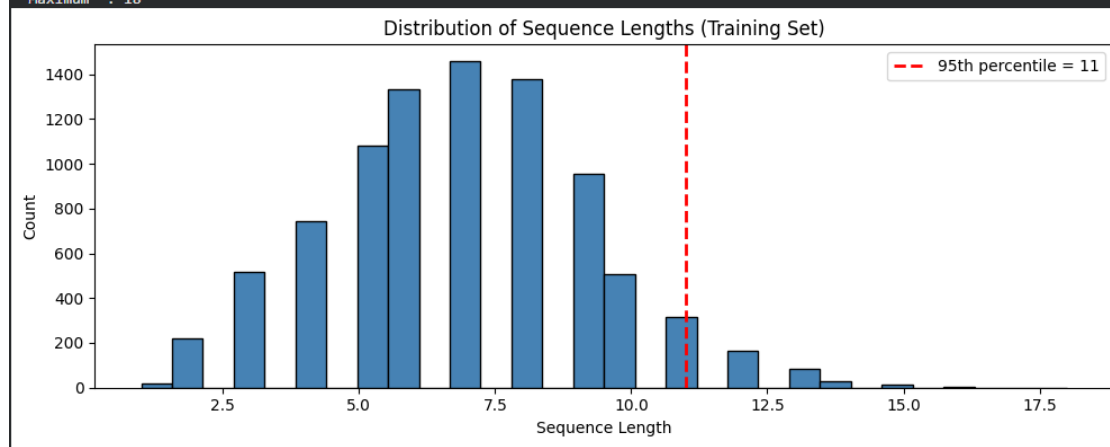


Figure 3: sequence length distribution histogram

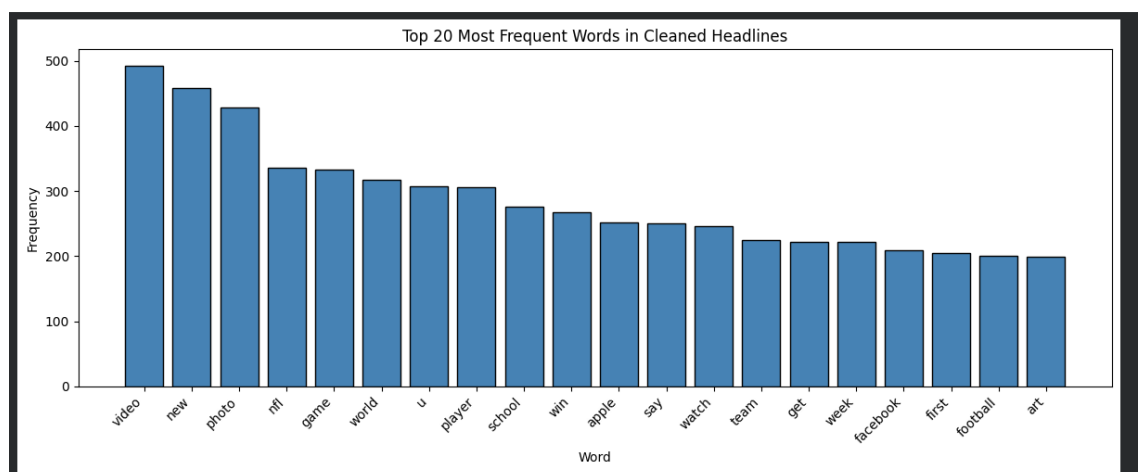


Figure 4: the overall word cloud of cleaned headlines



Figure 5: the per-category word cloud grid

3.2 Label Encoding

Category labels were integer-encoded using `sklearn.preprocessing.LabelEncoder`:

Integer Label	Category
0	CULTURE & ARTS
1	EDUCATION
2	MONEY
3	SPORTS
4	TECH

3.3 Model Architectures

Common Hyperparameters: EMBED_DIM = 128, VOCAB_SIZE = 10,000, MAX_LEN = 11, num_classes = 5.

Architecture A: SimpleRNN (Model 1)

A stacked SimpleRNN model with a trainable embedding layer:

Input → Embedding(10000, 128, input_length=11)
 → SimpleRNN(128, return_sequences=True, recurrent_dropout=0.2)
 → Dropout(0.3)
 → SimpleRNN(64, return_sequences=False)
 → Dropout(0.3)
 → Dense(128, activation='relu')
 → BatchNormalization()
 → Dropout(0.3)

→ Dense(5, activation='softmax')

Total Parameters: 1,334,725

Architecture B: Bidirectional LSTM (Model 2)

A stacked Bidirectional LSTM with a trainable embedding layer:

Input → Embedding(10000, 128, input_length=11)
 → Bidirectional(LSTM(128, return_sequences=True, recurrent_dropout=0.2))
 → Dropout(0.3)
 → Bidirectional(LSTM(64, return_sequences=False, recurrent_dropout=0.2))
 → Dropout(0.3)
 → Dense(128, activation='relu')
 → BatchNormalization()
 → Dropout(0.3)
 → Dense(5, activation='softmax')

Total Parameters: 1,725,189

The bidirectional wrapper allows the LSTM to process each sequence from both left-to-right and right-to-left, capturing richer contextual dependencies.

Architecture C: Bidirectional LSTM + Pretrained Word2Vec (Model 3)

Identical to Architecture B, but the embedding layer is initialised with a **frozen** pretrained Word2Vec matrix:

Input → Embedding(10000, 128, weights=[embed_matrix], trainable=False)
 → Bidirectional(LSTM(128, return_sequences=True, recurrent_dropout=0.2))
 → Dropout(0.3)
 → Bidirectional(LSTM(64, return_sequences=False, recurrent_dropout=0.2))

- Dropout(0.3)
- Dense(128, activation='relu')
- BatchNormalization()
- Dropout(0.3)
- Dense(5, activation='softmax')

Total Parameters: 1,725,189 (of which 1,280,000 are frozen)

Word2Vec Training Configuration:Ts

Parameter	Value
Algorithm	Skip-gram (sg=1)
Vector size	128 (matches EMBED_DIM)
Window size	7
Min word count	1
Training epochs	30
Vocabulary size	12,035
Embedding matrix shape	(10,000 × 128)
Tokens matched	9,998 / 9,999 (100.0%)

 **[SCREENSHOT PLACEHOLDER 3 — Model Summaries]** Paste here:

the model.summary() output for all three models (Cells 30, 32, 36). These show the layer names, output shapes, and parameter counts.

Model: "SimpleRNN_Model"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dropout (Dropout)	?	0
simple_rnn_1 (SimpleRNN)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)
batch_normalization (BatchNormalization)	?	0 (unbuilt)
dropout_2 (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Figure 6: `model.summary()` output for SimpleRNN_Model

Model: "LSTM_Model"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
bidirectional (Bidirectional)	?	0 (unbuilt)
dropout_3 (Dropout)	?	0
bidirectional_1 (Bidirectional)	?	0 (unbuilt)
dropout_4 (Dropout)	?	0
dense_2 (Dense)	?	0 (unbuilt)
batch_normalization_1 (BatchNormalization)	?	0 (unbuilt)
dropout_5 (Dropout)	?	0
dense_3 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Figure 7: `model.summary()` output for LSTM_Model

Model: "LSTM_Word2Vec_Model"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	1,280,000
bidirectional_2 (Bidirectional)	?	0 (unbuilt)
dropout_6 (Dropout)	?	0
bidirectional_3 (Bidirectional)	?	0 (unbuilt)
dropout_7 (Dropout)	?	0
dense_4 (Dense)	?	0 (unbuilt)
batch_normalization_2 (BatchNormalization)	?	0 (unbuilt)
dropout_8 (Dropout)	?	0
dense_5 (Dense)	?	0 (unbuilt)

Total params: 1,280,000 (4.88 MB)
Trainable params: 0 (0.00 B)
Non-trainable params: 1,280,000 (4.88 MB)

Figure 8: `model.summary()` output for `LSTM_Word2Vec_Model`

3.4 Training Setup

Setting	Value
Loss Function	Categorical Cross-Entropy
Optimizer	Adam
Metrics	Accuracy
Class Weighting	Balanced (sklearn.compute_class_weight)
Early Stopping	patience=6, monitor val_loss, restore best weights
Model Checkpoint	Saves best val_accuracy per architecture
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=3, min_lr=1e-5)
Hardware	NVIDIA GPU with TF dynamic memory growth

4. Experiments and Results

4.1 RNN vs. LSTM Performance

Summary of Training Results:

Model	Best Val Accuracy	Best Val Loss	Epochs Trained
Simple RNN	79.48%	0.7911	9
LSTM	83.90%	0.4869	9
LSTM + Word2Vec	84.01%	0.4851	15

Discussion: The SimpleRNN converged to early stopping after 9 epochs with a best validation accuracy of 79.48%. The vanishing gradient problem in standard RNNs limits the capacity to model even moderately long sequences, explaining its substantially lower performance. The BiLSTM models trained more stably and converged to significantly lower validation loss values (~0.487), confirming LSTMs' superior capabilities for sequential text modelling. The Word2Vec variant required more epochs (15) to reach its peak, but achieved marginally better validation performance.

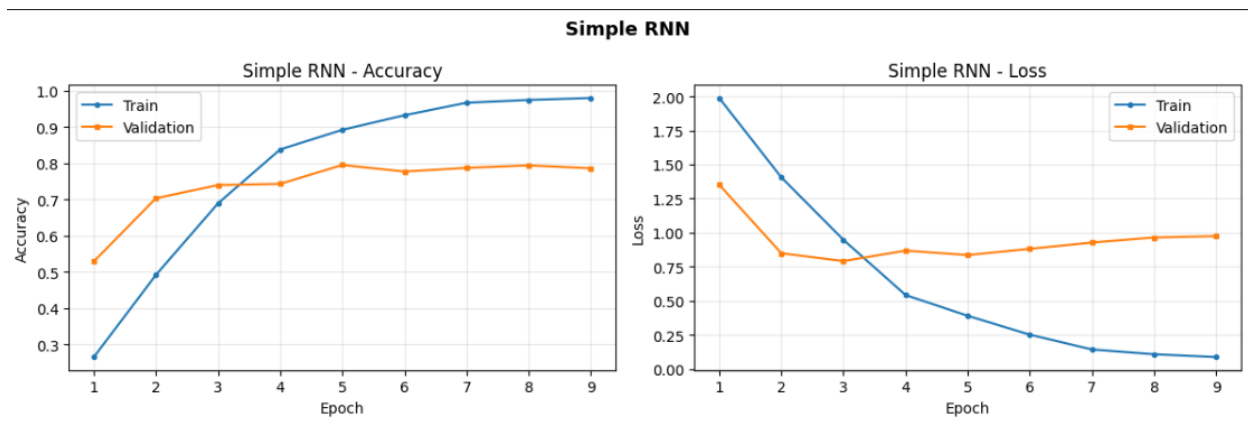


Figure 9: accuracy and loss training curves for Simple RNN

LSTM

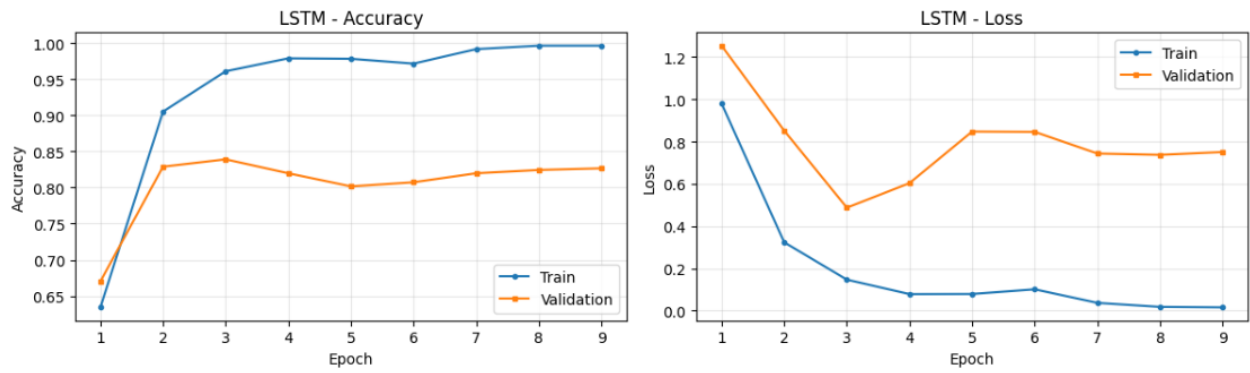


Figure 10: accuracy and loss training curves for LSTM

LSTM + Word2Vec

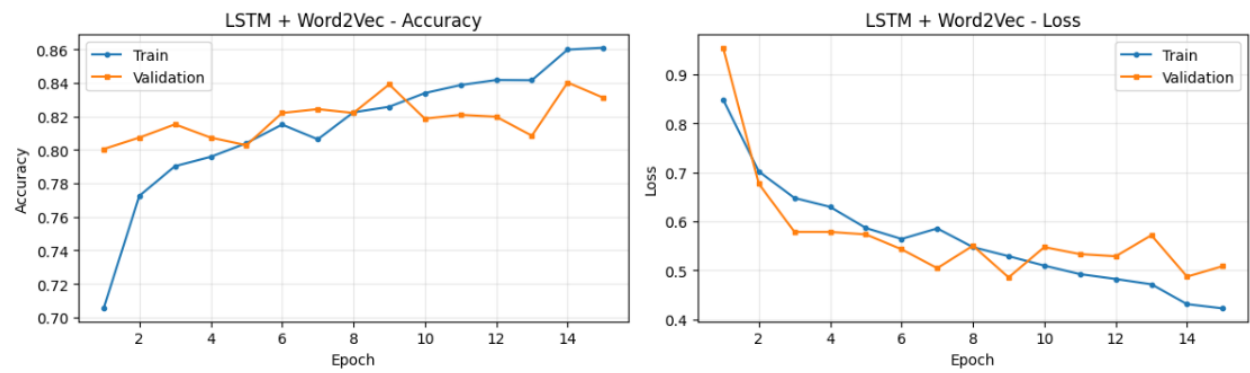


Figure 11: accuracy and loss training curves for LSTM+Word2vec

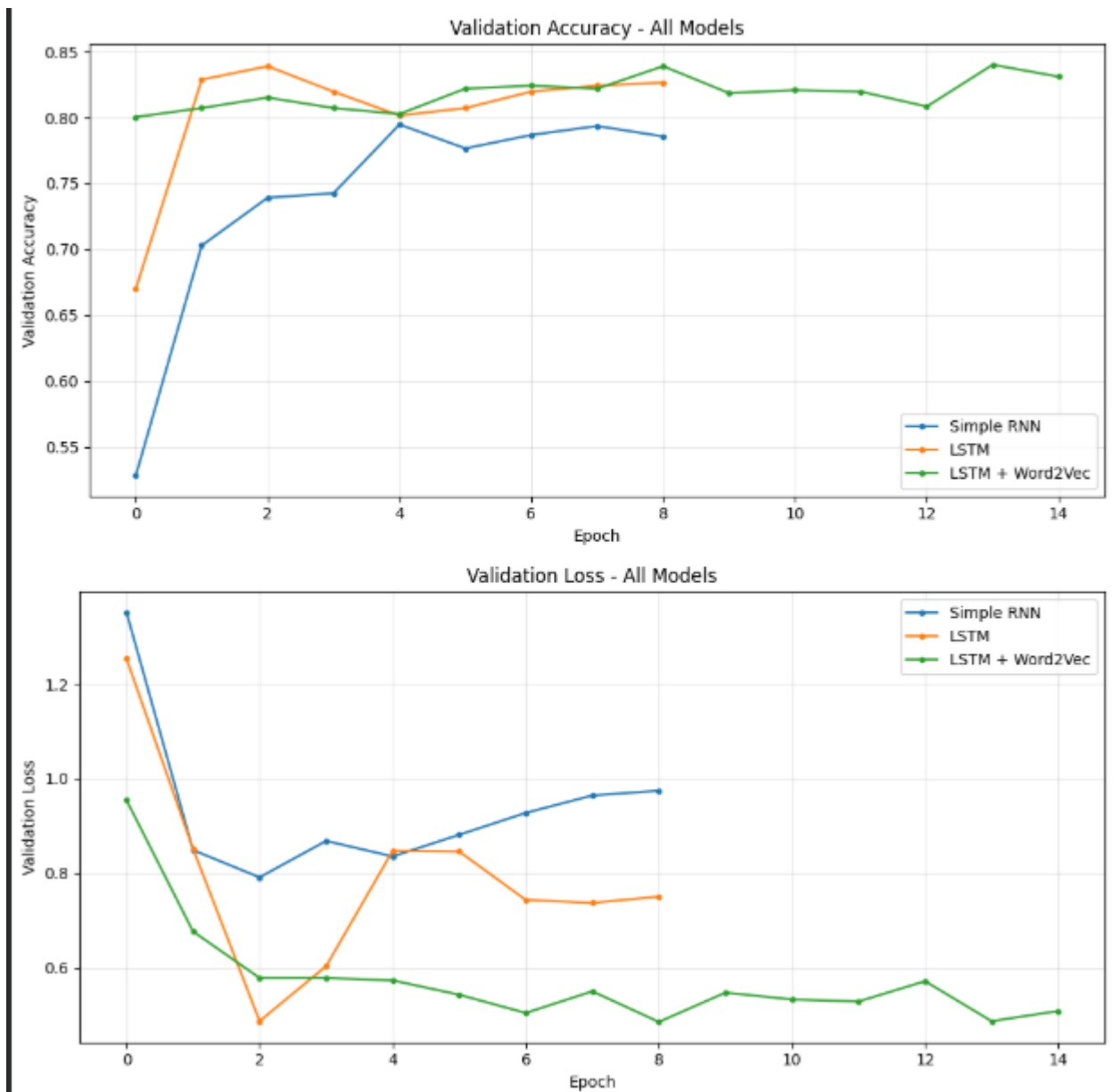


Figure 12:combined validation accuracy comparison chart for all three models

4.2 Computational Efficiency

Model	Parameters	Embedding Type	Approx. Relative Cost
Simple RNN	1,334,725	Trainable	Lowest
LSTM	1,725,189	Trainable	Higher
LSTM + Word2Vec	1,725,189	Frozen (1.28M)	Similar to LSTM

- Training was performed on a **NVIDIA GPU** with TensorFlow dynamic memory growth enabled (GPU:0 confirmed in output).
- LSTM layers require approximately **4× more operations** per timestep than SimpleRNN due to gating mechanisms.
- The frozen Word2Vec embedding layer reduces trainable parameters by 1,280,000 compared to Architecture B (making Architecture C faster per epoch despite running for more epochs overall).
- The use of Google Colab with GPU acceleration is strongly recommended; CPU-only training would be prohibitively slow for 15 epochs over 8,820 samples.

4.3 Training with Different Embeddings

Embedding Strategy	Architecture	Test Accuracy	Improvement
Random (trainable)	Simple RNN	71.79%	Baseline
Random (trainable)	BiLSTM	82.59%	+10.80 pp
Word2Vec skip-gram (frozen)	BiLSTM	83.90%	+1.31 pp

The pretrained Word2Vec embeddings contributed a **+1.31 percentage point** improvement over the equivalent BiLSTM with random initialisation. Near-complete token coverage (100%) ensured no representational gap. The skip-gram model trained with window=7 and 30 epochs is particularly suited for short headlines, as it captures wider co-occurrence contexts.

4.4 Model Evaluation on Held-Out Test Set

Simple RNN: Test Accuracy: 71.79%

Class	Precision	Recall	F1-Score	Support
CULTURE & ARTS	0.38	0.80	0.52	215
EDUCATION	0.48	0.63	0.55	203
MONEY	0.83	0.41	0.55	351
SPORTS	0.92	0.82	0.87	1,015
TECH	0.74	0.73	0.73	421
Macro Avg	0.67	0.68	0.64	2,205
Weighted Avg	0.78	0.72	0.73	2,205

LSTM: Test Accuracy: 82.59%

Class	Precision	Recall	F1-Score	Support
CULTURE & ARTS	0.66	0.80	0.72	215
EDUCATION	0.74	0.65	0.69	203
MONEY	0.89	0.66	0.76	351
SPORTS	0.87	0.94	0.90	1,015
TECH	0.81	0.80	0.80	421
Macro Avg	0.79	0.77	0.77	2,205
Weighted Avg	0.83	0.83	0.82	2,205

LSTM + Word2Vec: Test Accuracy: 83.90% (*Best Model*)

Class	Precision	Recall	F1-Score	Support
CULTURE & ARTS	0.71	0.80	0.75	215
EDUCATION	0.62	0.76	0.69	203
MONEY	0.83	0.73	0.77	351
SPORTS	0.93	0.92	0.93	1,015
TECH	0.83	0.79	0.81	421
Macro Avg	0.78	0.80	0.79	2,205
Weighted Avg	0.85	0.84	0.84	2,205

Key Observations:

- **SPORTS** is classified best across all models (F1 up to 0.93), primarily due to its dominant frequency in the dataset and distinctive domain vocabulary.
- **CULTURE & ARTS** and **EDUCATION** are the most challenging categories due to vocabulary overlap between them.
- The LSTM models show substantially improved recall for MONEY (0.66–0.73 vs. 0.41 for SimpleRNN), addressing the SimpleRNN's tendency to under-predict minority classes.
- The macro-average F1 improves from 0.64 (SimpleRNN) to 0.79 (LSTM + Word2Vec), a gain of 0.15 points, reflecting genuine improvement across all categories.

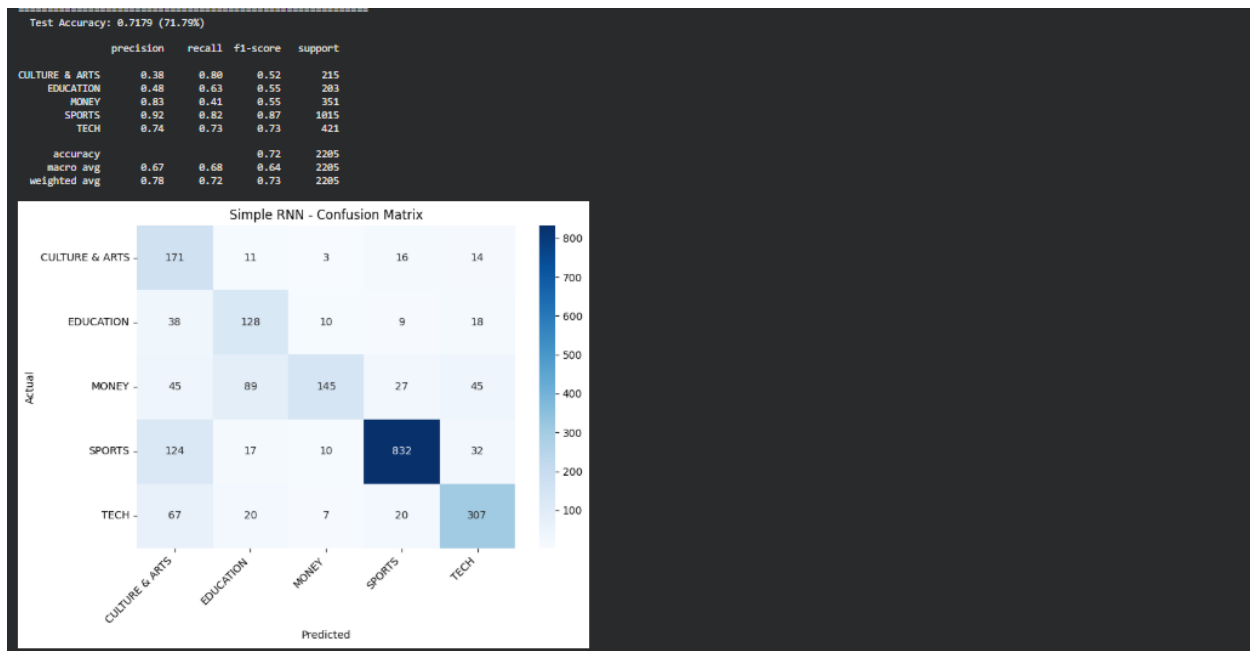


Figure 13: : the seaborn heatmap confusion matrices for Simple RNN

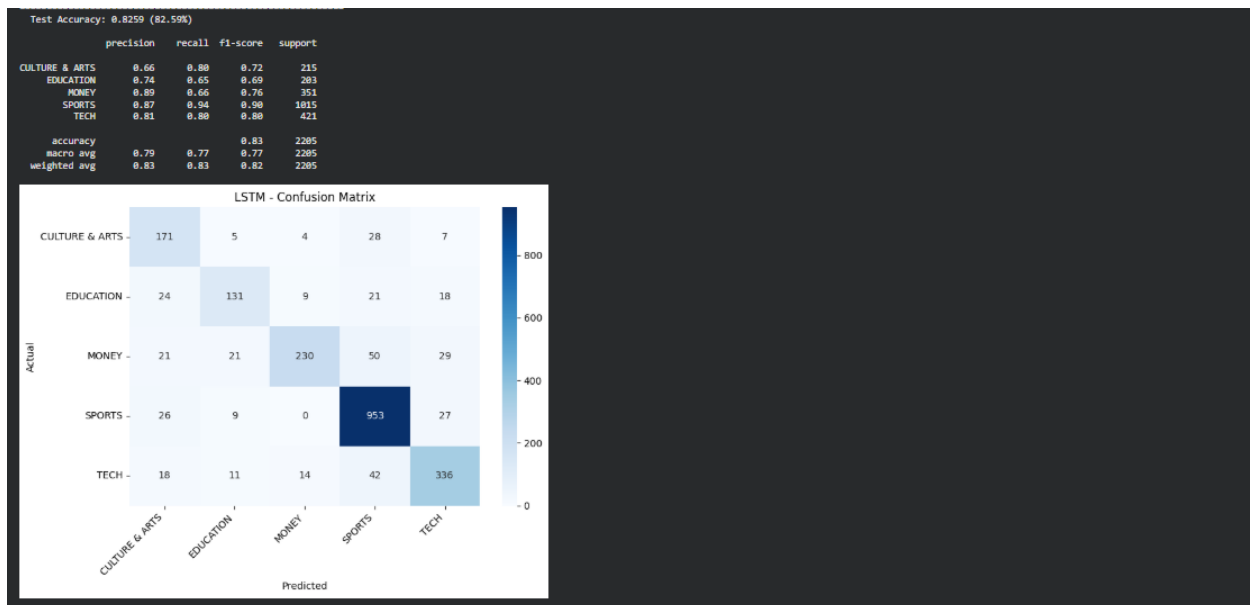


Figure 14: : the seaborn heatmap confusion matrices for LSTM

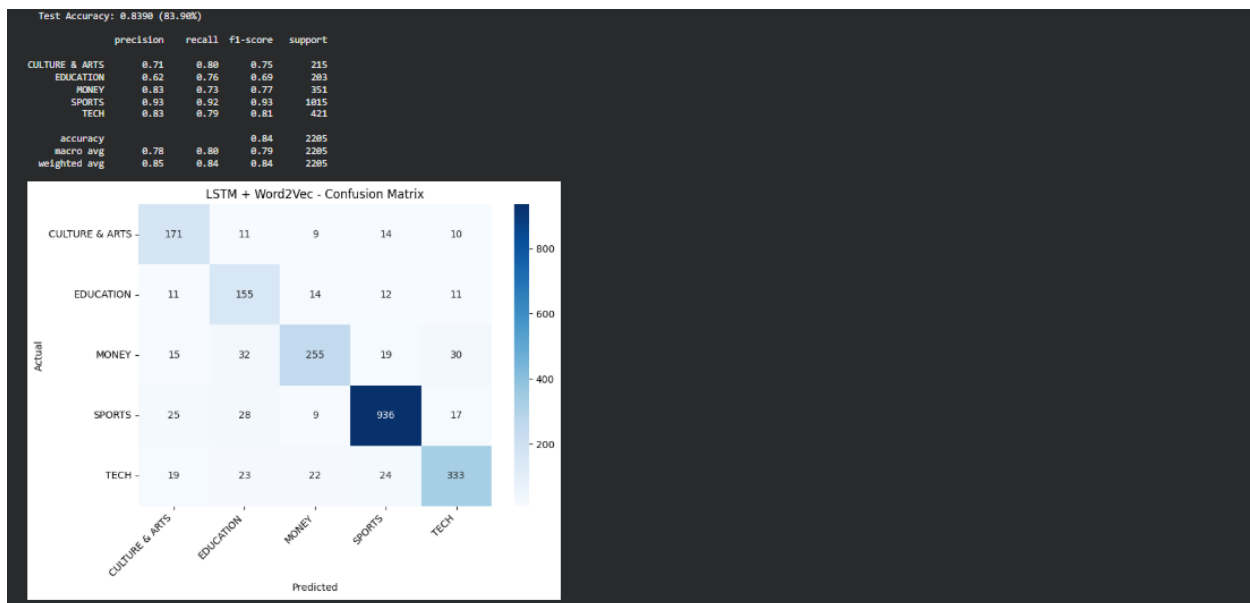


Figure 15: : the seaborn heatmap confusion matrices for LSTM+Word2vec

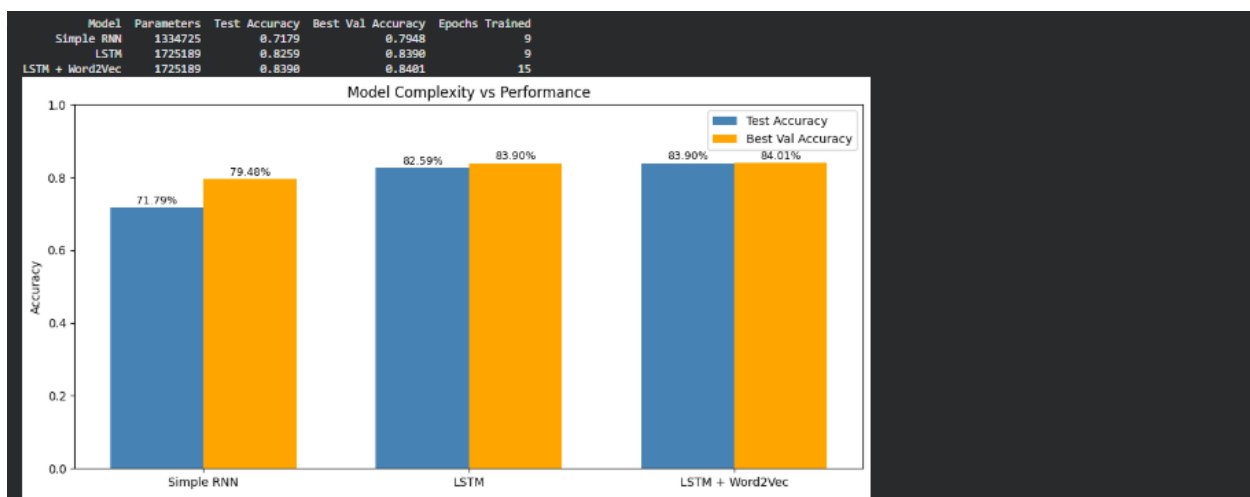


Figure 16: bar chart comparing Test Accuracy vs. Best Val Accuracy for all three models

Part 4: Error Analysis

Misclassification Summary (Best Model: LSTM + Word2Vec)

Metric	Value
Total Test Samples	2,205
Correctly Classified	1,850
Misclassified	355
Test Accuracy	83.90%

Sample Misclassified Headlines

Case	Processed Headline	True Category	Predicted
1	game boy childhood dream finally exists	TECH	CULTURE & ARTS
2	judicial negligence compound political negligence south carolina	EDUCATION	CULTURE & ARTS
3	tale autism spectrum friend amberdawn	CULTURE & ARTS	EDUCATION

```

Architecture : LSTM + Word2Vec
Total samples : 2,205
Errors : 355
Accuracy : 83.90%

Misclassification Examples (first 3):
=====

Case 1:
  Headline : game boy childhood dream finally exists
  True category : TECH
  Predicted : CULTURE & ARTS
-----

Case 2:
  Headline : judicial negligence compound political negligence south carolina
  True category : EDUCATION
  Predicted : CULTURE & ARTS
-----

Case 3:
  Headline : tale autism spectrum friend amberdawn
  True category : CULTURE & ARTS
  Predicted : EDUCATION
-----

```

Figure 17: Error Analysis Output

Failure Mode Analysis

Failure Mode	Explanation
Shared vocabulary across topics	Words like "game", "win", "show" appear in SPORTS, TECH, and CULTURE & ARTS — e.g., "game boy" is gaming (TECH) but contains CULTURE/SPORTS-associated words.
Very short processed sequences	After stopword/lemma processing, several headlines shrink to 2–3 tokens, giving the model insufficient context.
Category-neutral words	High-frequency generic terms ("new", "report", "star") carry little discriminative signal across categories.
Named entity corruption	Lemmatization can corrupt proper nouns — e.g., "Wills" (a person's name) becomes "will", shifting its semantic interpretation.

Design Decisions Already Incorporated

Modification	Motivation
Stacked Bidirectional LSTM	Captures context from both directions per token
Embedding size increased (64 → 128)	Denser, more expressive word representations
recurrent_dropout on LSTM layers	Regularises recurrent connections to reduce overfitting
BatchNormalization after Dense	Stabilises gradient flow during training
Balanced class weighting	Counteracts SPORTS-dominated class imbalance
Word2Vec skip-gram, window=7, 30 epochs	Better embeddings for short, domain-specific text

Potential Improvements

Improvement	Expected Benefit
GloVe 300d or Google News Word2Vec	Higher-quality external pretrained embeddings
Fine-tuning DistilBERT / BERT	Contextual subword embeddings, expected >90% accuracy
Synonym-based data augmentation	Helps underrepresented EDUCATION/CULTURE classes
Exclude lemmatizer for named entities	Prevents proper noun corruption
Hyperparameter search (Keras Tuner)	Optimal LSTM units, learning rate, batch size

Part 5: Interactive Prediction Interface (Gradio GUI)

A Gradio web application was built to enable **real-time news headline classification**. Users can enter any headline and select which model to use (Simple RNN, LSTM, or LSTM + Word2Vec). The interface returns the predicted category with its confidence percentage and a full breakdown of confidence scores across all five classes.

Implementation Summary:

Python code:

```
# Core function: clean headline → tokenise → pad → predict

def classify_headline(input_text: str, chosen_arch: str) -> tuple:

    processed = clean_text(input_text)

    encoded = text_tokenizer.texts_to_sequences([processed])

    seq_padded = pad_sequences(encoded, maxlen=MAX_LEN, ...)

    prob_scores = selected_net.predict(seq_padded)[0]

    return predicted_label, confidence_dict
```

Interface:

- **Input 1:** News headline textbox
- **Input 2:** Model dropdown (Simple RNN / LSTM / LSTM + Word2Vec)
- **Output 1:** Predicted category + confidence %
- **Output 2:** Confidence scores bar for all 5 classes

Launched via `demo_app.launch()` at <https://localhost:7860/> within the Colab environment.

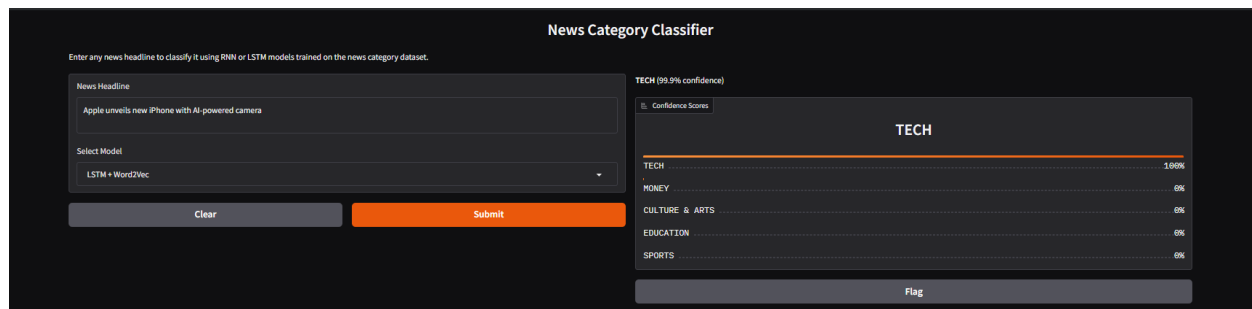


Figure 18: screenshot of the running Gradio interface in the browser

4. Conclusion and Future Work

Key Findings

The project tested three RNN-based models to classify news headlines into five different categories which included Sports Tech Money Culture Arts and Education:

1. **LSTM vastly outperforms SimpleRNN:** The test results demonstrate that LSTM outperforms SimpleRNN because LSTM maintains a +10.80 percentage point test accuracy improvement which shows that LSTM's gating system prevents vanishing gradients to perform better in sequential text processing tasks.
2. **Pretrained Word2Vec embeddings provide consistent improvement:** the frozen skip-gram embedding layer improved test accuracy by +1.31 pp (82.59% → 83.90%), with near-complete (100%) token coverage ensuring full semantic initialisation.
3. **Class imbalance is a persistent challenge:** SPORTS (~46% of data) achieved the highest F1 scores across all models (0.87–0.93). Minority classes such as EDUCATION and CULTURE & ARTS showed lower recall despite balanced class weights during training.
4. **Short-text classification requires richer representations:** with headlines averaging only 6.9 cleaned tokens, models have minimal context. This underscores the need for high-quality pretrained embeddings and motivates future adoption of transformer-based contextual models.

Limitations

- Word2Vec was trained on only 8,820 training headlines, which limits embedding quality compared to large external corpora.
- Vocabulary overlap between categories (e.g., EDUCATION vs. CULTURE & ARTS) creates inherent ambiguity that token-level sequence models cannot fully resolve without broader contextual understanding.
- SimpleRNN performance is fundamentally bounded by its inability to retain long-range dependencies, even within short sequences.

Future Work

- **Transformer models:** Fine-tuning DistilBERT or RoBERTa is expected to push accuracy beyond 90% through contextual subword embeddings.
- **External embeddings:** Replacing in-corpus Word2Vec with word2vec-google-news-300 or glove-wiki-gigaword-300 would improve representation quality.
- **Data augmentation:** Back-translation or synonym replacement could increase training diversity for minority classes.
- **Ensemble methods:** Combining predictions from SimpleRNN and LSTM through soft voting may reduce individual model errors.
- **Extended vocabulary:** Increasing VOCAB_SIZE from 10,000 to 20,000+ may capture more domain-specific terminology.

References

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.
- Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.
- Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45–50.
- Herald College Kathmandu. (2026). *6CS012 Final Portfolio Assessment 2026*. Unpublished assessment document.